LSM-дерево: индексирование под запись

Table of contents

LSM-дерево: индексирование под запись . . . 1
Строительные блоки . . . 2
SSTable . . . 2
Memtable . . . 2
Bloom Filter . . . 3
WAL . . . 6
Полная структура . . . 7
Пайплайн в движении . . . 8
Сложность операций . . . 8
Create / Update . . . 8
Read . . . 8
Delete . . . 8
Range Query . . . 9
Сравнение с B-tree и Hash . . . 9
LSM vs B-tree . . . 9
LSM vs Hash . . . 9
Практическое применение . . . 9

LSM-дерево: индексирование под запись

Первая структура, которую мы изучаем, это **LSM-дерево** (Log-Structured Merge Tree). LSM оптимизировано под **высокую скорость записи**. Это критично для систем, где запись доминирует над чтением: логи, метрики, IoT-телеметрия, временны́е ряды.

Идея в трёх шагах:

1. Записи сначала попадают в память в структуру **Memtable**.
2. При заполнении Memtable замораживается и сбрасывается на диск как отсортированный файл **SSTable**.
3. Периодически файлы сливаются фоновой **компакцией**, чтобы избежать фрагментации и удалить устаревшие версии.

LSM используется в Cassandra, RocksDB, LevelDB, HBase, Google Bigtable. Компромисс понятен: быстрая запись, но чтение может потребовать проверки нескольких файлов (read amplification).

Строительные блоки

SSTable

SSTable (Sorted String Table) это иммутабельный файл с парами `(ключ, значение)`, отсортированными по ключу.

Идея: данные пишутся крупными отсортированными блоками, что делает последовательное чтение быстрым и удобным для диапазонных запросов.

Что даёт:

- дешёвая запись на диск большими порциями;
- быстрые диапазонные чтения благодаря сортировке;
- предсказуемый I/O за счёт последовательного доступа.

Что внутри:

- набор отсортированных блоков данных;
- лёгкий индекс на блоки, чтобы не читать весь файл.

Go-псевдоинтерфейс и оценки:

```go
// SSTable: иммутабельное чтение по ключу и диапазону.
type SSTable interface {
    // Get: бинарный поиск по индексу + чтение блока.
    // O(log b + k), b — число блоков, k — размер блока.
    Get(key []byte) (value []byte, ok bool)

    // RangeScan: последовательное чтение по диапазону.
    // O(log b + r), r — число записей в диапазоне.
    RangeScan(start, end []byte, fn func(k, v []byte))
}
```

Memtable

Memtable это изменяемая структура в памяти для накопления записей перед сбросом в SSTable.

Смысл: быстрые записи в память, а на диск данные уходят порциями уже отсортированными.

Что внутри:

- упорядоченная структура в памяти, обычно skip list или RB-дерево;
- быстрая поддержка вставки и поиска по ключу.

Go-псевдоинтерфейс и оценки:

```go
// Memtable: изменяемые записи, сортировка в памяти.
type Memtable interface {
    // Put: вставка или обновление.
    // O(log m), m — число ключей.
    Put(key, value []byte)

    // Get: поиск по ключу.
    // O(log m).
    Get(key []byte) (value []byte, ok bool)

    // RangeScan: диапазонный проход.
    // O(log m + r).
    RangeScan(start, end []byte, fn func(k, v []byte))

    // Flush: упорядоченный сброс в новый SSTable.
    // O(m) на последовательную запись.
    Flush(builder SSTableBuilder)
}
```

Bloom Filter

Bloom Filter это компактная вероятностная структура, которая отвечает на один вопрос: «может ли ключ x лежать в этом множестве?». Ответа два: **точно нет** или **возможно, да**. Ложноположительные срабатывания возможны, ложноотрицательных не бывает никогда.

Для LSM это точно то, что нужно: у нас десятки SSTable на диске, и перед чтением каждого хочется быстро отсечь те, где искомого ключа заведомо нет, чтобы не тратить дисковые обращения.

Как устроен

Внутри фильтра всего две вещи:

- **битовый массив** B размера m, изначально весь нулевой — это и есть всё «хранилище» фильтра;
- k **независимых хеш-функций** $h_1, h_2, \ldots, h_k$. Каждая принимает ключ x и возвращает **позицию** (целое число) в диапазоне $[0, m)$ — то есть индекс в массиве B.

Самих ключей, ссылок на SSTable, счётчиков — ничего этого фильтр не хранит. Всё его «знание» о множестве закодировано в том, какие биты в B стоят в единицу.

На что именно возвращает хеш-функция. Обычно берут быструю 64-битную хеш-функцию (MurmurHash3, xxHash) и сводят её выход в нужный диапазон:

$$h_i(x) = \mathrm{hash}_i(x) \bmod m$$

или, если m это степень двойки, через дешёвую маску:

$$h_i(x) = \mathrm{hash}_i(x) \ (m-1)$$

То есть одна хеш-функция $\longrightarrow$ одна позиция $\longrightarrow$ один бит в B, который она «задевает».

Вставка `Add(x)`

Алгоритм буквально из трёх шагов:

1. посчитать k позиций $p_1 = h_1(x), \ldots, p_k = h_k(x)$;
2. выставить `B[p_i] = 1` для каждого i;
3. готово — ключ x «добавлен».

Заметим: если какой-то из этих битов уже был 1 (его раньше выставил другой ключ), мы просто оставляем его 1. Фильтр никак не различает «этот бит мой» и «этот бит выставил кто-то ещё» — именно отсюда в дальнейшем возьмутся ложные срабатывания.

Проверка `MightContain(x)`

Тоже три шага:

1. посчитать те же k позиций $p_1, \ldots, p_k$ (используя ровно те же хеш-функции, что и при вставке);
2. прочитать биты `B[p_1], ..., B[p_k]`;
3. вернуть ответ по правилу:
   - если **хотя бы один** из этих битов равен 0 $\longrightarrow$ ключа **точно нет** в множестве;
   - если **все** k битов равны 1 $\longrightarrow$ ключ **возможно** есть (может быть ложным срабатыванием).

Маленький пример на пальцах

Возьмём $m = 16$ (то есть битовый массив длины 16) и $k = 3$ хеш-функции. Начальное состояние:

```
index:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
B:      0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
```

Добавляем ключ `"apple"`. Пусть хеши вернули позиции $h_1 = 2$, $h_2 = 7$, $h_3 = 13$. Выставляем биты 2, 7, 13:

```
B:      0  0  1  0  0  0  0  1  0  0  0  0  0  1  0  0
```

Добавляем ключ `"banana"`. Пусть хеши дали $h_1 = 4$, $h_2 = 7$, $h_3 = 11$. Позиция 7 уже была 1 — и это нормально:

```
B:      0  0  1  0  1  0  0  1  0  0  0  1  0  1  0  0
```

Теперь проверяем:

- `MightContain("apple")` → смотрим позиции 2, 7, 13 → `1, 1, 1` → «возможно, есть». И это правда, мы `apple` добавляли.
- `MightContain("cherry")` → пусть хеши дали 1, 5, 9 → смотрим `B[1]=0` → **точно нет**. Останавливаемся на первом же нуле, дальше можно не проверять.
- `MightContain("grape")` → пусть хеши дали 2, 4, 11 → смотрим `1, 1, 1` → «возможно, есть». Но `grape` мы никогда не добавляли — это классическое **ложное срабатывание**: все нужные биты случайно выставили другие ключи (`apple` поставил бит 2, `banana` поставила биты 4 и 11).

Почему нет ложных отрицаний

Если ключ x был добавлен, то `Add(x)` точно выставил `B[h_1(x)] = ... = B[h_k(x)] = 1`. Никто эти биты обратно в 0 не снимает (фильтр иммутабельный, как и SSTable над которой он построен). Значит позже `MightContain(x)` увидит все k битов единицами и вернёт «возможно, есть». То есть «забыть» уже добавленный ключ фильтр не может.

Почему бывают ложные положительные

`MightContain(y)` для незнакомого ключа y вернёт «возможно, есть», если все k его битов случайно оказались выставленными другими ключами (как в примере с `grape`). Чем больше ключей мы уже запихнули в фильтр, тем больше в B единиц, и тем выше шанс такого совпадения.

Практический смысл

Для этой лекции достаточно трёх инженерных правил:

1. Чем **больше битов на ключ**, тем реже ложные срабатывания.
2. Чем **больше ключей запихнули в тот же фильтр**, тем он менее полезен: всё больше битов становятся единицами.
3. Слишком мало хешей плохо, но и слишком много тоже плохо: фильтр быстрее «забивается».

В реальных LSM-движках обычно не выводят параметры вручную на собеседовании у доски. Важно помнить порядок величин:

- около `10` бит на ключ это типичная практическая настройка;
- такой фильтр даёт примерно `1%` ложных срабатываний;
- ложное срабатывание не ломает корректность, а лишь приводит к одному лишнему чтению SSTable.

Оценка памяти тоже простая: `10` бит на ключ это примерно `1.25` байта на ключ. Значит SSTable с 10^6 ключей потребует около `1.25` МБ под Bloom Filter, и поэтому такие фильтры обычно держат в RAM.

Место в чтении LSM

Когда приходит `Get(key)`, LSM идёт по SSTable сверху вниз (от свежих уровней к старым). Перед тем как лезть в индекс и блоки данных конкретной SSTable:

1. считаются k хешей от `key`;

2. проверяются k битов в bloom-фильтре, который уже лежит в памяти;
3. если хоть один 0 — SSTable пропускается без единого дискового обращения;
4. если все 1 — делается обычное чтение индекса и блока, и это может быть ложная тревога.

Особенно сильно фильтр помогает на запросах по несуществующим ключам (типично в кэш-сценариях: «есть ли сессия с таким id?»). Без фильтра такой запрос спускается по всем уровням и делает L лишних чтений.

Тонкие места и ограничения

- **Нет удаления отдельных ключей.** Обычный bloom-фильтр не умеет «забывать». В LSM это не проблема: каждая SSTable иммутабельна, а при компакции создаётся новая SSTable и новый фильтр.
- **Нет поддержки range-запросов.** Фильтр отвечает только про точное равенство. Для диапазонных чтений используют отдельные структуры, например prefix Bloom или SuRF.
- **Независимость хешей.** На практике используют один быстрый хеш (MurmurHash, xxHash) и получают k функций через формулу $h_i(x) = h_a(x) + i \cdot h_b(x)$. Это корректно при больших m.
- **Стоимость ложного срабатывания.** Одно ложное срабатывание стоит одно «лишнее» чтение блока из SSTable. Поэтому 1% ложных срабатываний — это примерно 1% лишних чтений, что обычно терпимо.

Go-псевдоинтерфейс и оценки:

```go
// BloomFilter: вероятностная проверка наличия.
type BloomFilter interface {
    // Add: выставляет k битов, по одному на каждую хеш-функцию.
    // O(k).
    Add(key []byte)

    // MightContain: проверяет k битов.
    // Возвращает false → точно нет;
    // возвращает true  → возможно, есть (возможен ложный положительный ответ).
    // O(k).
    MightContain(key []byte) bool
}
```

WAL

WAL (Write-Ahead Log) это append-only журнал на диске. Каждая запись сначала фиксируется в WAL и только потом применяется к Memtable. При крахе процесса Memtable восстанавливается из WAL, так что подтверждённые записи не теряются.

Полная структура

LSM-дерево это конвейер: запись идёт быстро сверху вниз, а порядок поддерживается фоновыми слияниями.

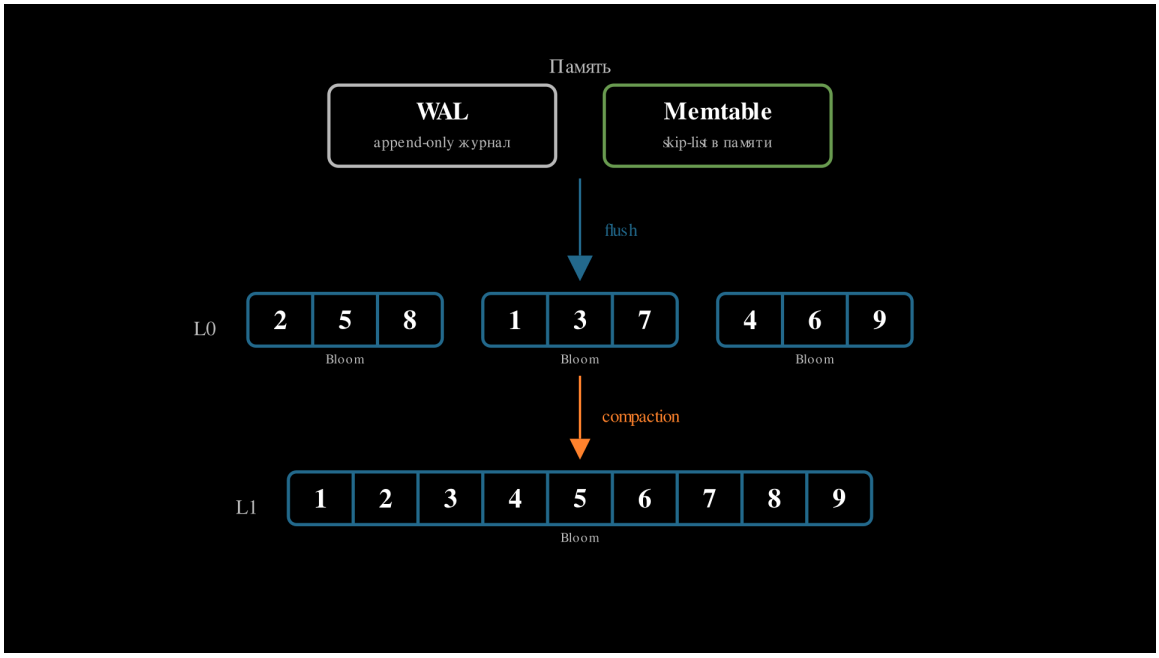

Figure 1: Пайплайн LSM: Memtable в памяти, несколько SSTable на L0, слитый SSTable на L1.

Основные части:

1. WAL — журнал для надёжности, каждая запись сначала фиксируется последовательно.
2. Memtable — быстрая запись и сортировка в памяти.
3. Immutable Memtable — замороженная память, ожидающая сброса.
4. Набор SSTable на диске — отсортированные файлы, каждый с Bloom Filter.

Как работает запись:

- клиент пишет `(ключ, значение)`;
- запись попадает в WAL и Memtable (быстро, в памяти);
- при переполнении Memtable замораживается и сбрасывается в новый SSTable (последовательный I/O).

Как работает чтение:

- поиск идёт от самых свежих структур к старым: Memtable → Immutable Memtable → SSTable L0 → SSTable L1, ...;
- перед чтением SSTable проверяется Bloom Filter, чтобы не читать лишнее с диска;
- результат берётся из первой найденной версии.

Как поддерживается порядок и место:

- на диске постепенно накапливаются SSTable с пересекающимися ключами;

- фоновая компакция сливает их в более крупные и упорядоченные файлы;
- при слиянии старые версии и tombstone-записи выкидываются, освобождая место.

Пайплайн в движении

Анимация показывает две последовательные Memtable-пачки, которые сбрасываются в два SSTable на L0, после чего компакция сливает их в один отсортированный SSTable на L1.

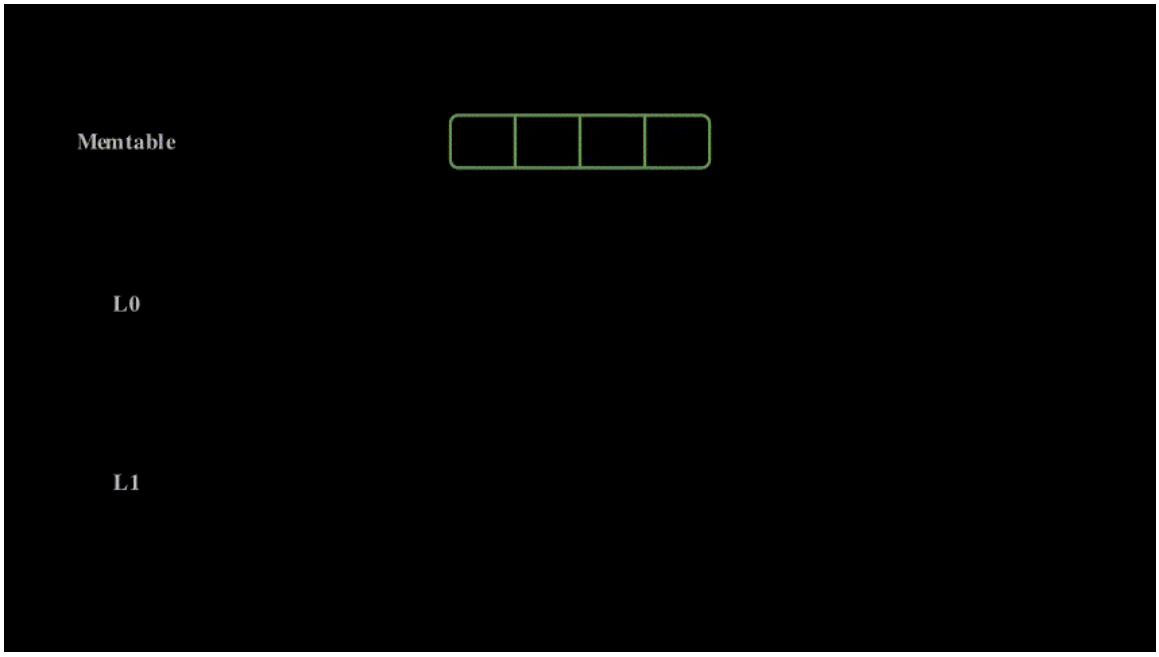

Figure 2: Конвейер LSM: вставки в Memtable, flush на L0, компакция на L1.

Сложность операций

Create / Update

$$O(\log m)$$

- запись в WAL: $O(1)$ последовательная;
- вставка в Memtable: $O(\log m)$, где m это число ключей в Memtable;
- periodic flush: $O(m)$ последовательной записи, амортизируется по всем вставкам.

Read

$$O(\log m + L \cdot \log b)$$

- поиск в Memtable: $O(\log m)$;
- по каждому уровню из L Bloom Filter отсекает ненужные SSTable;
- в оставшемся SSTable бинарный поиск по индексу блоков: $O(\log b)$.

Delete

$$O(\log m)$$

- удаление это вставка специальной tombstone-записи в Memtable;
- физическое удаление случится во время ближайшей компакции, которая пройдёт по соответствующему уровню.

Range Query

$$O(\log m + R)$$

- находим левые границы во всех актуальных структурах;
- сливаем отсортированные потоки на лету, получая R записей в результате.

Сравнение с B-tree и Hash

LSM vs B-tree

- **Чтение:** B-tree за $O(\log_B n)$ попадает в нужный лист; LSM может обойти Memtable и несколько SSTable (Bloom Filter ускоряет, но не гарантирует).
- **Запись:** LSM обычно быстрее — append-only в Memtable и последовательный flush, без read-modify-write узлов.
- **Стабильность:** B-tree даёт предсказуемую задержку; у LSM бывают всплески из-за компакции.
- **Итого:** B-tree для OLTP и low-latency чтений; LSM для write-heavy и time-series.

LSM vs Hash

- hash даёт $O(1)$ по ключу, но теряет сортировку и не поддерживает range-сканы;
- LSM хорош для диапазонных запросов и потоков событий.

Практическое применение

Хорошо подходит:

- частые вставки и обновления по ключу (write-heavy нагрузки);
- пакетные записи и стриминг событий;
- диапазонные запросы по ключу;
- чтение свежих данных, где важна скорость записи и допустима амортизированная задержка.

Плохо подходит:

- случайные чтения по множеству ключей без кэша (много SSTable);
- низкая и стабильная задержка чтения в p99 (компакция и пересечения);
- частые операции удаления, если критично немедленно освободить место.

Типичные системы на этой идее:

Система	Зачем LSM
RocksDB / LevelDB	базовый KV-движок от Google/Meta, встраивается в десятки систем

Система	Зачем LSM
Cassandra	write-heavy нагрузки, time-series, распределённые логи
HBase	Hadoop-экосистема, аналитика поверх HDFS
Google Bigtable	семинальная система, откуда LSM пошёл в prod
InfluxDB	time-series: постоянный поток метрик, редкие обновления